

Quantitative Options Arbitrage

Arbitrage-Free Surface Calibration, Portfolio Optimization, and Execution

Systematic Trading Research

May 25, 2026

Abstract

This report develops a mathematical framework for statistical relative-value options trading in an A-share 4×15 expiry-strike matrix. The framework converts Level 2 (L2) order-book data into an actionable mispricing score vector $\mathbf{W}$ through arbitrage-free surface calibration, liquidity screening, and low-latency predictive inference. It then maps the resulting alpha estimates into a constrained portfolio optimization problem with explicit Greek exposure controls and direct bid-ask execution logic for 1–10 minute mean-reversion horizons.

1 Pricing Matrix Transformation and Scoring

The objective of this phase is to evaluate the current market state across N available option contracts (where $N \leq 60$ for a 4×15 A-share options matrix) and generate a score vector $\mathbf{W} \in \mathbb{R}^{N'}$ over the tradable universe, representing the degree to which each option deviates from the theoretical TP_n surface.

1.1 Liquidity Filtering and Effective Price Matrix

Raw L2 order-book data contains multiple levels of bids and asks, but not all quoted depth is economically tradable. In this implementation, the L2 feed is treated as a fixed-depth order-book snapshot with periodic updates and multiple bid-ask levels. Let $i \in \{1, 2, \dots, N\}$ index the available options. At any tick t , we define the *effective* ask price P_i^A and effective bid price P_i^B using volume thresholds. Given the actionable latency bounds, we accept the first level (L1) if the volume $v(\cdot)$ exceeds 5 contracts. If not, we sweep the first two levels (L1 + L2) if their combined volume exceeds 5. Otherwise, the option is deemed untradable.

The effective ask P_i^A is formally defined as:

$$P_i^A = \begin{cases} a_{1,i} & \text{if } v(a_{1,i}) > 5 \\ a_{2,i} & \text{if } v(a_{1,i}) \leq 5 \text{ and } [v(a_{1,i}) + v(a_{2,i})] > 5 \\ \emptyset & \text{otherwise (illiquid)} \end{cases} \quad (1)$$

A symmetric function defines the effective bid P_i^B using $b_{1,i}$ and $b_{2,i}$. When the second level is required, $a_{2,i}$ and $b_{2,i}$ should be interpreted as conservative executable boundary prices rather than guaranteed best-offer or best-bid fills; a production implementation may replace them with depth-weighted execution prices. We drop any contract i where $P_i^A = \emptyset$ or $P_i^B = \emptyset$, reducing our universe to N' tradable contracts. For these tradable contracts, we define the market mid-price vector $\mathbf{M} \in \mathbb{R}^{N'}$:

$$M_i = \frac{P_i^A + P_i^B}{2} \quad (2)$$

We then construct the current state matrix $\mathbf{X} \in \mathbb{R}^{N' \times F}$, where F is the number of features. Each row $\mathbf{x}_i$ represents the state of contract i ; later, $\mathcal{X}_t$ denotes the full expiry-strike state tensor used by the activation layer:

$$\mathbf{X} = \begin{bmatrix} S_t & K_1 & \tau_1 & P_1^B & P_1^A & \sigma_{IV,1} \\ S_t & K_2 & \tau_2 & P_2^B & P_2^A & \sigma_{IV,2} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ S_t & K_{N'} & \tau_{N'} & P_{N'}^B & P_{N'}^A & \sigma_{IV,N'} \end{bmatrix} \quad (3)$$

1.2 Arbitrage-Free Fair Value Surface Calibration

Let $\{(K_i, T_j)\}_{i=1, \dots, 15; j=1, \dots, 4}$ denote the discrete strike-maturity grid, where $K_1 < K_2 < \dots < K_{15}$ represents the sorted cross-section of active strike prices for a given maturity slice. Let M_{ij} be the observed midprice of the European option with strike K_i and maturity T_j , and let S_t denote the contemporaneous underlying price. We seek a smooth call-price surface

$$\hat{C} : \mathbb{R}_+ \times \mathbb{R}_+ \rightarrow \mathbb{R}_+, \quad (K, T) \mapsto \hat{C}(K, T),$$

which fits the observed quotes while satisfying static no-arbitrage constraints. We parameterize the surface by a coefficient vector $\theta \in \mathbb{R}^d$, for example through a tensor-product spline basis,

$$\hat{C}(K, T; \theta) = \sum_{m=1}^d \theta_m \phi_m(K, T),$$

where $\{\phi_m\}_{m=1}^d$ are fixed basis functions. The calibration problem is then formulated as the following optimization framework:

$$\begin{aligned} \min_{\theta \in \mathbb{R}^d} \quad & \sum_{j=1}^4 \sum_{i=1}^{15} w_{ij} \left(\hat{C}(K_i, T_j; \theta) - M_{ij} \right)^2 + \lambda \mathcal{R}(\theta) \\ \text{s.t.} \quad & \partial_K \hat{C}(K, T; \theta) \leq 0, \quad \forall (K, T), \\ & \partial_{KK} \hat{C}(K, T; \theta) \geq 0, \quad \forall (K, T), \\ & \partial_T \hat{C}(K, T; \theta) \geq 0, \quad \forall (K, T), \\ & \max(0, S_t - K e^{-rT}) \leq \hat{C}(K, T; \theta) \leq S_t, \quad \forall (K, T), \\ & \hat{C}(0, T; \theta) = S_t, \quad \forall T, \\ & \lim_{K \rightarrow \infty} \hat{C}(K, T; \theta) = 0, \quad \forall T. \end{aligned} \quad (4)$$

Here $w_{ij} > 0$ are confidence weights, $r \geq 0$ is the risk-free interest rate, and $\mathcal{R}(\theta)$ is a smoothness penalty, such as an H^2 -seminorm or a discrete curvature regularizer.

Due to the structural configuration of the SSE options market, the strike grid is inherently non-uniform. This irregularity stems from multi-tiered baseline strike intervals that expand as the underlying price shifts across thresholds, compounded by ex-dividend adjustments where historical strikes are multiplied by precision fractions—causing uneven intervals to coexist alongside newly listed standard contracts.

To guarantee a strictly butterfly arbitrage-free surface across an irregular grid, the continuous convexity constraints must be enforced using an asymmetric, non-uniform finite-difference stencil. Let $\hat{C}_{ij} := \hat{C}(K_i(T_j), T_j; \theta)$ denote the call price evaluated at strike $K_i(T_j)$ for maturity T_j . Because the active strike grid is non-uniform and varies dynamically across maturity slices T_j due to corporate actions and ex-dividend adjustments, discrete calendar arbitrage cannot be naively enforced by comparing matching strike indices. Instead, calendar no-arbitrage must be evaluated continuously via $\partial_T \hat{C}(K, T; \theta) \geq 0$, or projected onto a unified reference grid of strike prices $\{K_i^{\text{ref}}\}$. Assuming such a

unified reference projection or maturity-invariant strike grid subset, a mathematically rigorous set of linear inequality constraints for the discrete finite grid implementation is defined as:

$$\begin{aligned} \frac{\hat{C}_{i+1,j} - \hat{C}_{ij}}{K_{i+1} - K_i} &\leq 0, & i = 1, \dots, 14, j = 1, \dots, 4, \\ \frac{\hat{C}_{i+1,j} - \hat{C}_{ij}}{K_{i+1} - K_i} &\geq \frac{\hat{C}_{ij} - \hat{C}_{i-1,j}}{K_i - K_{i-1}}, & i = 2, \dots, 14, j = 1, \dots, 4, \\ \hat{C}(K_i^{\text{ref}}, T_{j+1}; \theta) &\geq \hat{C}(K_i^{\text{ref}}, T_j; \theta), & i = 1, \dots, 15, j = 1, \dots, 3. \end{aligned} \quad (5)$$

Rearranging the non-uniform convexity constraint into an explicit linear form suitable for standard high-speed primal-dual interior point solvers transforms the second condition in (5) into:

$$\left(\frac{1}{K_i - K_{i-1}}\right) \hat{C}_{i-1,j} - \left(\frac{1}{K_{i+1} - K_i} + \frac{1}{K_i - K_{i-1}}\right) \hat{C}_{ij} + \left(\frac{1}{K_{i+1} - K_i}\right) \hat{C}_{i+1,j} \geq 0 \quad (6)$$

The resulting arbitrage-free fair-value vector is obtained by evaluation on the live market grid:

$$\hat{\mathbf{P}}^{\text{fair}} = (\hat{C}(K_i, T_j; \theta^*))_{i,j},$$

where θ^* solves (4) under the linear boundaries defined above. Because the 4×15 dimensions of the grid remain small, this calibration executes as a deterministic, constrained quadratic program or projected least-squares problem, safely tracking within the tight 10ms engineering target.

1.3 Unified Tree-Based Expected Return Framework (W)

To capture non-linear mean-reversion dynamics without compromising the 10ms latency constraint, the calculation of the mispricing score vector $\mathbf{W}$ is split into a two-stage architecture: (1) an instantaneous microstructural spread gate, and (2) a single unified gradient-boosted decision tree model $\mathcal{T}$ that scores all active contracts in the underlying options matrix. Unlike an instrument-specific design, this pooled model shares information across strikes, maturities, and contract types while still producing contract-level expected alpha over the 1–10 minute horizon.

1.3.1 Stage 1: Latency-Constrained Screening Gate

At each L2 tick update, we calculate the instantaneous raw mispricing $Z_i = \hat{P}_i^{\text{fair}} - M_i$. Before any model prediction is performed, we also impose an instrument-specific liquidity and execution bound $\lambda_i > 0$ that absorbs unmodeled transaction fees, queue risk, and slippage concessions. Contract i is routed to the predictive engine if and only if it is liquid enough and the executable edge clears this bound:

$$\mathcal{G}_i = \begin{cases} 1 & \text{if } \hat{P}_i^{\text{fair}} - P_i^A > \lambda_i \quad (\text{Candidate Underpriced}) \\ 1 & \text{if } P_i^B - \hat{P}_i^{\text{fair}} > \lambda_i \quad (\text{Candidate Overpriced}) \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

Contracts where $\mathcal{G}_i = 0$ bypass model inference entirely, setting $W_i = 0$ automatically to conserve clock cycles inside the high-frequency execution loop. This ordering ensures the liquidity bound is applied before the tree model predicts executable alpha.

1.3.2 Stage 2: Batched Unified Tree Inference

All contracts where $\mathcal{G}_i = 1$ are compiled into a compact dynamic batch matrix $\Xi_t \in \mathbb{R}^{K \times F}$, where $K \leq N'$, and evaluated by the shared model $\mathcal{T}$ in a single high-speed inference pass. Rather than predicting mid-price adjustments, $\mathcal{T}$ directly maps current market states to an expected executable return over the target horizon Δt .

We define the *Immediate Execution Gap* D_i as the directional distance between the theoretical surface and the executable order book layer required to capture it:

$$D_i = \begin{cases} \hat{P}_i^{\text{fair}} - P_i^A & \text{if } \hat{P}_i^{\text{fair}} > P_i^A \quad (\text{Edge after buying the Ask}) \\ P_i^B - \hat{P}_i^{\text{fair}} & \text{if } \hat{P}_i^{\text{fair}} < P_i^B \quad (\text{Edge after selling the Bid}) \end{cases} \quad (8)$$

Because the unified model must distinguish contracts across the strike-maturity grid, each active contract is represented by an augmented feature vector that includes the execution edge, underlying state, surface-location variables, and local microstructure:

$$\xi_{i,t} = [D_i, S_t, (K_i - S_t), \tau_i, \mathbb{I}_{\{\text{Put}\}}, \Delta S_t^{(1m)}, \text{OFI}_{i,t}, s_i, \sigma_{IV,i}]^\top \quad (9)$$

where S_t is the underlying spot price, $(K_i - S_t)$ denotes moneyness distance, τ_i is the time-to-maturity, $\mathbb{I}_{\{\text{Put}\}} \in \{0, 1\}$ captures contract type, $\Delta S_t^{(1m)}$ represents short-term underlying momentum, $\text{OFI}_{i,t}$ is the localized order flow imbalance, $s_i = P_i^A - P_i^B$ is the formally defined executable bid-ask spread, and $\sigma_{IV,i}$ is the implied volatility. These cross-sectional variables replace the implicit contract identity that would otherwise be embedded in separate per-instrument models.

To satisfy high-frequency requirements, the model $\mathcal{T}$ is deployed using high-performance inference acceleration:

- **GPU-Native Pipeline:** Where features reside on-chip, $\mathcal{T}$ is evaluated using parallelized GPU tree-predictors (e.g., CUDA-accelerated XGBoost), eliminating host-to-device memory copy overhead.
- **Compiled CPU Pipeline:** Where features are processed on host, $\mathcal{T}$ is compiled via an asymmetric tree compiler (e.g., Treelite) into optimized C conditional structures, executing single-row traversals in nanosecond bounds (typically 50–300 ns depending on tree depth and CPU L1 cache locality).

1.3.3 Score Vector Generation

The unified model outputs the predicted executable alpha, $\hat{\alpha}_i = \mathcal{T}(\xi_{i,t})$, which represents the expected profit conditional on crossing the spread. Since the liquidity bound has already been enforced by the pre-inference gate, the final mispricing score vector $\mathbf{W}$ is constructed as follows:

$$W_i = \begin{cases} \hat{\alpha}_i & \text{if } \hat{P}_i^{\text{fair}} > P_i^A \\ -\hat{\alpha}_i & \text{if } \hat{P}_i^{\text{fair}} < P_i^B \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

This vector $\mathbf{W}$ flows directly into the portfolio optimization routine, with the sign convention chosen so that $W_i > 0$ favors long exposure and $W_i < 0$ favors short exposure. The objective therefore maximizes positions with statistically validated forward edge that has already passed the liquidity bound.

2 Portfolio Optimization and Execution Risk Controls

With the contract-level expected alpha vector $\mathbf{W} \in \mathbb{R}^{N'}$ dynamically generated at each Level 2 (L2) tick update, the trading engine follows a four-stage pipeline: opportunity activation, portfolio optimization, microstructural execution, and automated unwinding. This organization separates the decision of *when* to refresh the fair-value surface from the decision of *how much* to trade and *how* to execute the resulting orders.

Let $\Theta \in \mathbb{R}^{N'}$ represent the target portfolio allocation vector, where $\Theta_i > 0$ denotes a long position and $\Theta_i < 0$ denotes a short position. The live inventory vector is denoted by Θ_{current} .

2.1 Formal Portfolio Optimization Problem

Before execution routing, the contract-level expected alpha estimates are converted into a risk-controlled portfolio weight vector $\mathbf{w} \in \mathbb{R}^{N'}$. Let $\boldsymbol{\mu}$ denote the expected executable return vector, $\boldsymbol{\Sigma}$ denote the short-horizon return covariance matrix, and let $\boldsymbol{\Delta}$, $\boldsymbol{\Gamma}$, and $\mathbf{V}$ denote the cross-sectional Delta, Gamma, and Vega exposure vectors. The formal mean–variance portfolio program is:

$$\max_{\mathbf{w} \in \mathbb{R}^{N'}} \boldsymbol{\mu}^\top \mathbf{w} - \lambda_r \mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w} \quad (11)$$

$$\text{s.t. } \boldsymbol{\Delta}^\top \mathbf{w} = 0, \quad (12)$$

$$|\boldsymbol{\Gamma}^\top \mathbf{w}| \leq b_\Gamma, \quad (13)$$

$$|\mathbf{V}^\top \mathbf{w}| \leq b_V, \quad (14)$$

$$\sum_{i=1}^{N'} |w_i| \leq L. \quad (15)$$

Here $\lambda_r > 0$ controls risk aversion, b_Γ and b_V bound convexity and volatility exposure, and L limits gross leverage. This quadratic program establishes the portfolio-level objective before the lower-latency LP implementation in Section 2.3 converts target exposures into executable inventory adjustments.

2.2 Selective Opportunity Activation Layer

The first stage determines whether the market state is sufficiently informative to justify a new surface calibration and model-inference pass. The system does not perform full fair-value reconstruction on every market update. Instead, it first computes a lightweight activation score using the latest option-state tensor. Let

$$\mathcal{X}_t \in \mathbb{R}^{4 \times 15 \times d}$$

denote the option state tensor at time t , containing midprice, spread, depth, and auxiliary microstructure features.

The activation score is defined as

$$\Gamma_t = g(\mathcal{X}_t, S_t),$$

where $g(\cdot)$ is a fast screening function that identifies potentially exploitable local distortions before the more expensive calibration and optimization stages are invoked.

2.2.1 Activation Score Construction

The score is constructed from four components:

$$\Gamma_t = w_1 R_t + w_2 L_t + w_3 Q_t + w_4 E_t \quad (16)$$

where

- R_t : surface residual intensity,
- L_t : local shape violation score,
- Q_t : execution quality score,
- E_t : expected edge estimate.

The residual intensity measures aggregate deviation from the previously calibrated fair-value surface:

$$R_t = \sum_{i,j} \omega_{ij} |M_{ij,t} - \hat{C}_{ij,t-1}| \quad (17)$$

where $M_{ij,t}$ is the observed option midprice and $\hat{C}_{ij,t-1}$ is the last calibrated fair value.

To identify local mispricing clusters rather than global market motion, a shape-violation statistic is introduced:

$$L_t = \sum_{i,j} \left(\max(0, \Delta_K M_{ij,t}) + \max(0, -\Delta_{KK} M_{ij,t}) + \max(0, -\Delta_T M_{ij,t}) \right) \quad (18)$$

which measures violations of monotonicity and convexity constraints. Here Δ_K , Δ_{KK} , and Δ_T denote the discrete strike-slope, strike-convexity, and maturity-monotonicity operators corresponding to the no-arbitrage inequalities in (5).

Execution quality suppresses signals generated by unstable quotes. To prevent wide spreads and quote flickering in illiquid out-of-the-money (OTM) options from globally deactivating the calibration trigger, Q_t is computed using volume-weighted and delta-weighted averages:

$$Q_t = \frac{1}{1 + \alpha \bar{s}_t^w + \beta \sigma_t^w} \quad (19)$$

where the weighted average spread $\bar{s}_t^w$ and short-horizon quote volatility σ_t^w are defined across the active option contracts as:

$$\bar{s}_t^w = \frac{\sum_{i=1}^{N'} \omega_i s_{i,t}}{\sum_{i=1}^{N'} \omega_i}, \quad \sigma_t^w = \frac{\sum_{i=1}^{N'} \omega_i \sigma_{i,t}^{\text{micro}}}{\sum_{i=1}^{N'} \omega_i} \quad (20)$$

Here, $s_{i,t}$ is the individual contract spread, $\sigma_{i,t}^{\text{micro}}$ is the short-horizon quote volatility, and the weights $\omega_i = |\Delta_i| \cdot v(M_{i,t})$ are delta-volume weights prioritizing liquid near-the-money (NTM) contracts, ensuring noisy, un-tradable tail contracts do not paralyze system activation.

Finally, expected executable edge is estimated by

$$E_t = \sum_{i,j} \omega_{ij} \max \left(0, \frac{|M_{ij,t} - \hat{C}_{ij,t-1}| - c_{ij,t}}{c_{ij,t}} \right) \quad (21)$$

with $c_{ij,t}$ representing transaction and slippage costs.

2.2.2 Calibration Trigger Rule

Full surface calibration is triggered only when

$$\Gamma_t > \tau_{\text{enter}} \quad (22)$$

otherwise the system reuses the previous fair-value surface

$$\hat{C}_t = \hat{C}_{t-1}.$$

This event-driven design converts continuous evaluation into sparse activation events. When the trigger fires, the system refreshes the fair-value surface, regenerates $\mathbf{W}$, and passes the updated score vector into the optimization model in Section 2.3. Otherwise, the previous surface is reused and the system avoids unnecessary computation during low-signal regimes.

2.3 The Multi-Greek Constrained Optimization Model

The primary objective is to maximize expected mispricing capture across the tradable options matrix while enforcing strict cross-sectional risk limits. Over the 1–10 minute target regression horizon, the framework controls directional spot exposure through delta constraints, limits volatility exposure through vega constraints, and imposes a minimum theta-carry requirement.

To eliminate non-differentiable absolute-value terms from the capital and inventory allocations, we decompose the target position vector into its non-negative long and short components. This keeps the optimization compatible with deterministic, low-latency primal-dual interior-point methods:

$$\Theta = \Theta^+ - \Theta^-, \quad \text{where } \Theta^+ \geq \mathbf{0}, \Theta^- \geq \mathbf{0} \quad (23)$$

When the activation layer authorizes a refresh, the complete Linear Programming (LP) formulation is solved as follows:

$$\max_{\Theta^+, \Theta^-} (\Theta^+ - \Theta^-)^\top \mathbf{W} \quad (24)$$

$$\text{s.t.} \quad -\epsilon_\Delta \leq (\Theta^+ - \Theta^-)^\top \Delta \leq \epsilon_\Delta \quad (25)$$

$$-\epsilon_\nu \leq (\Theta^+ - \Theta^-)^\top \nu \leq \epsilon_\nu \quad (26)$$

$$(\Theta^+ - \Theta^-)^\top \vartheta \geq -\epsilon_\vartheta \quad (27)$$

$$(\Theta^+ + \Theta^-)^\top \mathbf{C} \leq C_{\max} \quad (28)$$

where the terms are defined as follows:

- $\Delta, \nu, \vartheta \in \mathbb{R}^{N'}$ represent the cross-sectional vectors of option Delta ($\partial C / \partial S$), Vega ($\partial C / \partial \sigma$), and Theta ($\partial C / \partial t$) evaluated at the contemporaneous tick state.
- ϵ_Δ and ϵ_ν define strict risk tolerances for net portfolio delta and vega exposure, ensuring structural immunization against underlying spot shocks and parallel volatility surface shifts.
- $\epsilon_\vartheta \geq 0$ represents the portfolio Theta risk budget (maximum allowable net time-decay rate). Bounding the aggregate time decay by $-\epsilon_\vartheta$ protects the high-frequency strategy from severe premium erosion during quiet regimes while preventing optimization infeasibility when the optimal alpha allocation is heavily weighted toward long-option positions.
- $\mathbf{C} \in \mathbb{R}^{N'}$ is the vector of capital requirements per contract (margin usage for short legs and premium deployment for long legs), bounded globally by the strategy's maximum allocation limit $C_{\max}$.

An important mathematical feature of this LP formulation is the natural complementarity at the optimal solution. Because both the objective coefficients W_i and the constraints are linear, and any simultaneous positive allocation to both Θ_i^+ and Θ_i^- strictly increases capital consumption $(\Theta^+ + \Theta^-)^\top \mathbf{C}$ without changing the directional net position or Greek exposures, the optimal solution naturally satisfies the complementarity condition $\Theta_i^+ \cdot \Theta_i^- = 0$ for all i . This guarantees that the optimization remains a convex, highly efficient Linear Program (LP) solvable in sub-millisecond bounds, without requiring non-convex binary variables.

2.4 Microstructural Execution Mapping

Once the optimization engine computes the optimal target vector $\Theta^* = \Theta^{+*} - \Theta^{-*}$, the execution layer computes the required net position change vector $\Delta\Theta \in \mathbb{R}^{N'}$ relative to the live inventory vector Θ_{current} :

$$\Delta\Theta = \Theta^* - \Theta_{\text{current}} \quad (29)$$

The portfolio LP determines the desired inventory target, but immediate order-book depth does not constrain the total inventory vector Θ^* . It constrains only the incremental trade submitted at the current tick. To mitigate execution latency and maximize fills before surface anomalies vanish, position adjustments bypass passive queue placement entirely. Instead, the routing engine issues aggressive limit orders mapped to the effective liquidity boundaries:

- **For $\Delta\Theta_i > 0$ (Accumulate Long / Cover Short):** The execution engine routes an aggressive buy limit order priced exactly at the effective ask price P_i^A . The trade execution size q_i is capped strictly based on the active liquidity price level:

$$q_i = \begin{cases} \min(\Delta\Theta_i, v(a_{1,i})) & \text{if } P_i^A = a_{1,i} \\ \min(\Delta\Theta_i, v(a_{1,i}) + v(a_{2,i})) & \text{if } P_i^A = a_{2,i} \end{cases} \quad (30)$$

- **For $\Delta\Theta_i < 0$ (Liquidate Long / Initiate Short):** The execution engine routes an aggressive sell limit order priced exactly at the effective bid price P_i^B . The trade execution size q_i is capped symmetrically:

$$q_i = \begin{cases} \min(|\Delta\Theta_i|, v(b_{1,i})) & \text{if } P_i^B = b_{1,i} \\ \min(|\Delta\Theta_i|, v(b_{1,i}) + v(b_{2,i})) & \text{if } P_i^B = b_{2,i} \end{cases} \quad (31)$$

2.5 Regression Horizons and Automated Unwinding

The alpha signal $\mathbf{W}$ targets short-term statistical anomalies with an expected mean-reversion half-life of 1–10 minutes. Positions are actively managed by an automated state machine that mandates immediate unwinding under three exhaustive conditions:

1. **Statistical Convergence:** If the absolute alpha score $|W_i|$ of an open position drops below an empirical exit threshold α_{exit} , the contract is flagged as having converged to the fair TP_n surface and is aggressively liquidated to realize profits.
2. **Temporal Hard Cutoff:** If a position remains open beyond the soft time limit $t_{\text{soft}} = 15$ minutes, the exit threshold α_{exit} is tightened progressively to accelerate liquidation. Any position that remains in inventory beyond the hard time limit $t_{\text{max}} = 30$ minutes is mandatorily unwound. This rule limits tail-risk accumulation, model-regime drift, and adverse premium decay.
3. **Greek Boundary Violations:** If unexpected underlying spot or volatility jumps push the aggregate portfolio into significant violation of constraints (25), (26), or (27), a high-priority micro-rebalancing routine is initialized to restore neutrality, prioritizing the liquidation of contracts displaying the lowest risk-to-alpha ratios.

All unwinding actions use the aggressive cross-spread routing logic outlined in Section 2.4 to prioritize immediate execution.